MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING

B.Tech (CE)
Semester: III
Course: Data Structure and Algorithms
Academic Year: 2026-27

Presented by: Ishansh Thada - 70022500334
Rajat Agrawal - 70022500345

Index

- Introduction
- Problem Statement
- Objectives
- Data Structures Used
- Linked List & Tree
- Navigation Stack & Recent Files Queue
- Time Complexity Analysis
- Flow of the Code
- Output
- Conclusion & Future Scope

Introduction

What is the CLI File Explorer?

- A terminal-based simulation of an operating system's directory and file manager.
- Implemented entirely in C using dynamic pointers and memory allocation without external database libraries.

Core Components:

- General Tree: Models folders containing subfolders and files.
- LIFO Stack: Keeps track of visited parent folders for fast step-back navigation.
- FIFO Queue: Logs accessed files in chronological order.

Problem Statement

1. Difficulty in Organizing Folders: Fixed-size arrays cannot easily represent nested folders with different levels of depth.

2. Difficulty in Folder Navigation: Moving between folders requires keeping track of the previous directory to support the Go Back operation.

3. Lack of File Access History: There is no simple way to track recently opened files in the order they were accessed.

Objectives

Main objectives are:

1. To implement an interactive in-memory file manager in C using DSA.
2. To model multi-level hierarchical folders using a Left-Child / Right-Sibling Tree.
3. To implement an array-based Stack for parent directory backtracking.
4. To implement a linked-list Queue for chronological recent-file logging.
5. To demonstrate real-time interaction between multiple data structures in one system.
6. To provide search using recursive Depth-First Search (DFS) traversal.
7. To create an intuitive, menu-driven CLI interface.

Data Structures Used

Data Structure	Application in Project
Linked List	Stores files and folders within a directory
Tree	Represents the hierarchical folder structure
Stack	Maintains previous folder navigation
Queue	Stores recently accessed files

- Linked List: Allows dynamic insertion and deletion of files and folders.
- Tree: Represents parent-child relationships between directories and their contents.
- Stack: Follows LIFO (Last In, First Out), allowing users to return to the previous folder.
- Queue: Follows FIFO (First In, First Out), maintaining files in the order they were accessed.

Data Structures Used — Linked List & Tree

Linked List (File and Folder Management):

- Stores multiple files and folders inside a directory.
- Each node contains the name and type of the file or folder.
- Uses pointers to connect different nodes.
- Allows dynamic memory allocation.

Tree Structure (Directory Hierarchy):

- Represents the relationship between folders and their contents.
- Each folder can contain multiple files and subfolders.
- child points to the first item inside a folder.
- next points to the next item in the same directory.

Stack, Queue & Core Operations

Stack (Folder Navigation):

- Used to maintain the history of previously visited directories.
- Push operation stores the current folder before opening another folder.
- Pop operation retrieves the previous folder when Go Back is selected.

Queue (Recently Accessed Files):

- Maintains a record of opened files.
- Uses front and rear pointers.

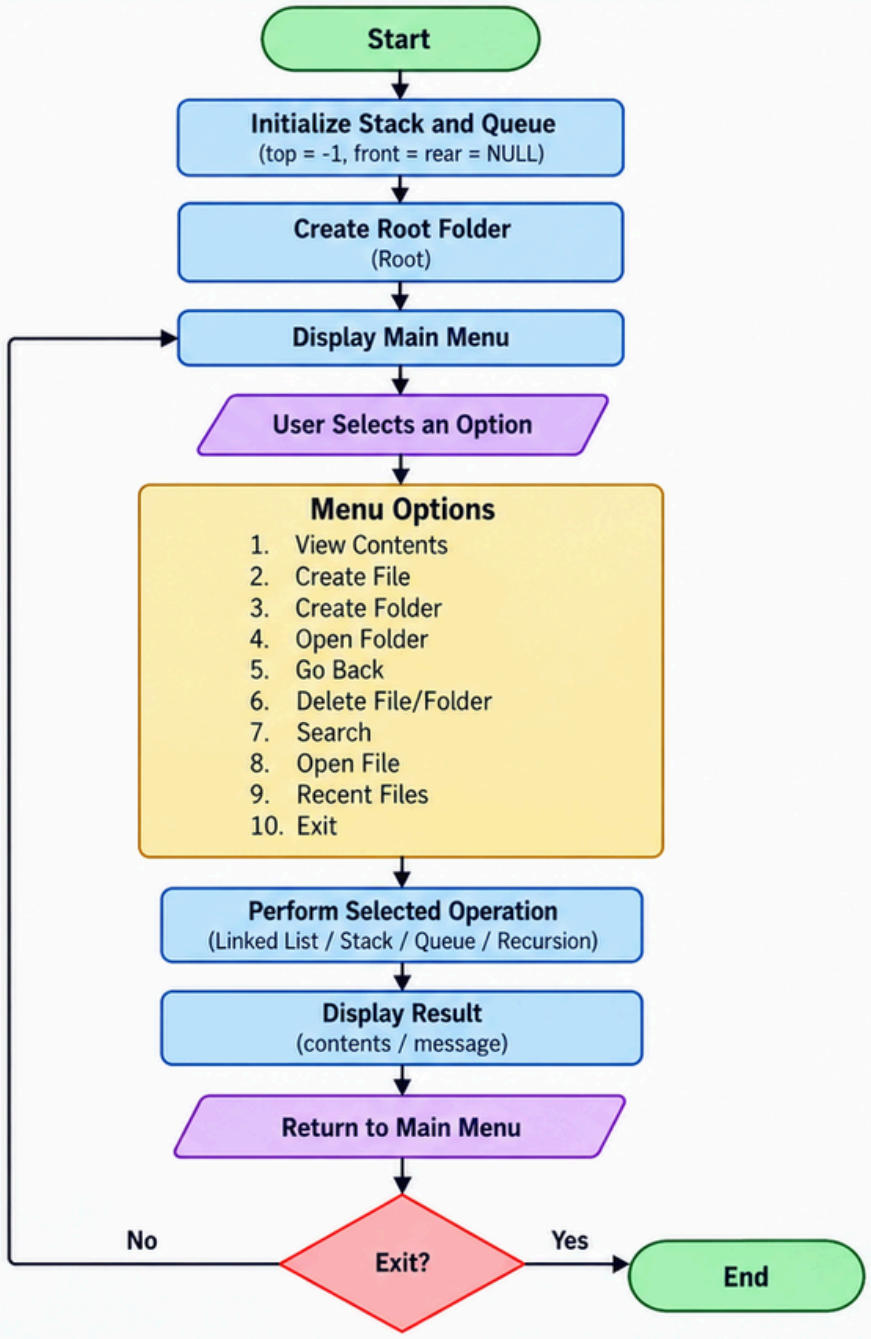
Key Operations Summary:

- addNode() – Creates and inserts a new file or folder.
- display() – Displays the contents of the current directory.
- deleteNode() – Removes a file or folder.
- search() – Searches for a file or folder recursively.
- addRecent() – Adds an accessed file to the queue.
- showRecent() – Displays recently accessed files.

Time Complexity

Operation	Time Complexity
createNode()	O(1)
addNode()	O(n)
display()	O(n)
addRecent()	O(1)
showRecent()	O(r)
search()	O(N)
deleteNode()	O(n)
Open Folder	O(n)
Go Back	O(1)
Open File	O(n)

Flow of the Code
File Explorer Using Data Structures



Data Structure Flow (Key Operations)

Directory Structure (Linked List / Tree)

Folder Node
name
type (file/folder)
child (first item)
next (next item)

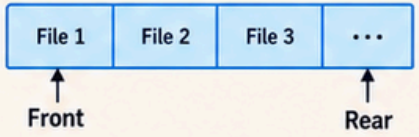
- Each node stores the name and type of the file or folder.
- 'child' points to the first item inside a folder.
- 'next' points to the next item in the same directory.
- Represents nested folders using a tree structure implemented with linked lists.

Folder Navigation (Stack)

Push current folder
(before opening a child folder)
↓
Pop previous folder
(on Go Back)

- Stores previously visited folders.
- Follows LIFO (Last In, First Out).
- Helps in navigating back to the parent directory.
- Implemented using an array.

Recent Files (Queue)



- Stores files in the order they are opened.
- Add newly opened file at the rear.
- Uses front and rear pointers.
- Follows FIFO (First In, First Out).
- Displays recently accessed files.

Search (Recursion)

Check items in current directory
↓
If item is a folder, search inside it recursively

- Compares the name with each file and folder.
- If the item is a folder, the search function is called recursively.
- Continues searching through all nested folders until the item is found.

Key Operations

- **addNode()** – Creates a new file or folder.
- **display()** – Lists contents of the current directory.
- **deleteNode()** – Removes a file or folder.
- **search()** – Finds a file or folder by name (recursively).
- **addRecent()** – Records an opened file in the queue.

Key Points

- Menu-driven C program.
- Simulated file system stored in memory.
- Multiple data structures work together.
- Uses linked list (tree), stack, queue and recursion.
- Demonstrates practical application of DSA concepts.

Data Structure Flow:

- File System → Linked List / Tree
- ↓
- Create File/Folder → Insert New Node
- ↓
- Open Folder → Push Current Folder into Stack
- ↓
- Go Back → Pop Previous Folder from Stack
- ↓
- Open File → Add File to Queue
- ↓
- Search → Recursively Traverse Nested Folders
- ↓
- Display / Delete → Traverse and Update Linked List
-

Output

```
========== FILE EXPLORER ==========
Current Folder: Pictures

1. View Contents
2. Create File
3. Create Folder
4. Open Folder
5. Go Back
6. Delete File/Folder
7. Search
8. Open File
9. Recent Files
10. Exit

Enter choice: 5
Moved back.

========== FILE EXPLORER ==========
Current Folder: Root

1. View Contents
2. Create File
3. Create Folder
4. Open Folder
5. Go Back
6. Delete File/Folder
7. Search
8. Open File
9. Recent Files
10. Exit

Enter choice: 1

Contents:
[Folder] Documents
[Folder] Pictures
[Folder] Projects

========== FILE EXPLORER ==========
Current Folder: Root

1. View Contents
2. Create File
3. Create Folder
4. Open Folder
5. Go Back
6. Delete File/Folder
7. Search
8. Open File
9. Recent Files
10. Exit

Enter choice: 4
Enter folder name: Projects
Folder opened.
```

```
========== FILE EXPLORER ==========
Current Folder: Root

1. View Contents
2. Create File
3. Create Folder
4. Open Folder
5. Go Back
6. Delete File/Folder
7. Search
8. Open File
9. Recent Files
10. Exit

Enter choice: 1

Contents:
[Folder] Documents
[Folder] Pictures
[Folder] Projects

========== FILE EXPLORER ==========
Current Folder: Root

1. View Contents
2. Create File
3. Create Folder
4. Open Folder
5. Go Back
6. Delete File/Folder
7. Search
8. Open File
9. Recent Files
10. Exit

Enter choice: 4
Enter folder name: Pictures
Folder opened.

========== FILE EXPLORER ==========
Current Folder: Pictures

1. View Contents
2. Create File
3. Create Folder
4. Open Folder
5. Go Back
6. Delete File/Folder
7. Search
8. Open File
9. Recent Files
10. Exit

Enter choice: 1
Folder is empty.
```

Conclusion & Future Scope

Conclusion

- The project demonstrates the practical implementation of Linked List, Tree, Stack, and Queue.
- These data structures work together to manage files, folders, and navigation history.
- Recursive searching allows files to be located across nested directories.

Future Scope

- Add support for file renaming.
- Add recursive deletion of folders and their contents.
- Implement file sorting based on name or type.
- Develop a graphical user interface.
- Integrate actual system file management operations.

Thank you